

چکیده

در این مقاله، یک قاعده یادگیری مبتنی بر همبستگی در سطح تیزه با دقت میلی‌ثانیه‌ای ارائه و تحلیل شده است. زمان‌بندی تیزه‌های پیش‌سیناپسی و پس‌سیناپسی از طریق یک "شدت اتصال نورونی یادگیری" نامتقارن بر وزن‌های سیناپسی تأثیر می‌گذارد. یک معادله برای توصیف فرایند یادگیری استخراج و برای یک مدل نورونی تصادفی حل شده است. نتایج مقاله نشان می‌دهد که همبستگی بین تیزه‌های ورودی و خروجی به تثبیت یادگیری کمک کرده و باعث نرمال‌سازی وزن‌های سیناپسی و نرخ شلیک نورون می‌شود. (که البته نتیجه‌ی ما این نبود!)

مقدمه

یادگیری هب یکی از اصول مهم در علوم اعصاب است که بیان می‌کند اگر یک نورون پیش‌سیناپسی به‌طور مکرر باعث فعال شدن نورون پس‌سیناپسی شود، قدرت اتصال سیناپسی بین آن‌ها افزایش می‌یابد. این نوع یادگیری برای شکل‌گیری نقشه‌های حسی و تنظیم اتصالات نورونی اهمیت زیادی دارد. مدل‌های قبلی بیشتر بر نرخ شلیک نورون‌ها تمرکز داشتند که فعالیت متوسط نورون‌ها را در بازه‌های زمانی طولانی نشان می‌دهد. اما در مقیاس زمانی کوتاه، فعالیت نورون‌ها شامل تیزه‌های الکتریکی سریع است. تحقیقات نشان داده است که زمان‌بندی دقیق این تیزه‌ها در مقیاس میلی‌ثانیه‌ای نقش مهمی در پردازش اطلاعات و تنظیم سیناپس‌ها دارد.

این مقاله با هدف ارائه یک قاعده یادگیری هب در سطح تیزه‌ها نوشته شده است که تأثیر همبستگی زمانی دقیق بین تیزه‌های پیش‌سیناپسی و پس‌سیناپسی را بررسی می‌کند. این مدل، برخلاف مدل‌های سنتی، اطلاعات زمانی تیزه‌ها را در نظر می‌گیرد و توانایی آن در تنظیم سیناپس‌ها و شکل‌گیری ساختارهای پایدار تحلیل شده است.

بدنه اصلی

این مقاله یک قاعده یادگیری جدید بر پایه مدل هب ارائه می‌دهد که به جای تمرکز بر نرخ شلیک، روی تیزه‌های نورونی و زمان‌بندی دقیق آن‌ها تأکید دارد. در این مدل، تغییر وزن سیناپسی نه تنها به وقوع تیزه‌های پیش‌سیناپسی و پس‌سیناپسی، بلکه به همبستگی زمانی بین آن‌ها وابسته است. در ادامه، ساختار کلی مدل توضیحات داده شده است:

۱. قاعده یادگیری هب در سطح تیزه‌ها

مدل پیشنهادی شامل سه مؤلفه اصلی است:

- **اثر تیزه ورودی:** هر تیزه پیش‌سیناپسی (S_{in}) تغییر کوچکی در وزن سیناپسی ایجاد می‌کند.
 - **اثر تیزه خروجی:** هر تیزه پس‌سیناپسی (S_{out}) به‌طور همزمان وزن تمام سیناپس‌های ورودی را تغییر می‌دهد.
 - **اثر همبستگی زمانی:** همبستگی زمانی بین پالس‌ها با استفاده از یک "پنجره یادگیری" ($W(\Delta t)$) تعیین می‌شود که زمان‌بندی دقیق پالس‌ها را در نظر می‌گیرد.
- معادله کلی تغییر وزن سیناپسی به این صورت تعریف شده است:

$$\Delta J_i(t) = h \int_t^{t+T} [w_{\text{in}} S_{\text{in}}(t') + w_{\text{out}} S_{\text{out}}(t')] dt' + h \int_t^{t+T} \int_t^{t+T} W(t''-t') S_{\text{in}}(t'') S_{\text{out}}(t') dt'' dt'.$$

در این معادله:

نرخ یادگیری که یک پارامتر کوچک است. h ، t وزن سیناپسی در زمان $J_i(t)$

مقادیر ثابت که تأثیر تیزه‌های ورودی و خروجی را تعیین می‌کنند. $w_{\text{in}}, w_{\text{out}}$

پنجره یادگیری که همبستگی زمانی بین پالس‌ها را تنظیم می‌کند. $W(\Delta t)$

۲. پنجره یادگیری (Learning Window)

پنجره یادگیری $W(\Delta t)$ یک تابع نامتقارن از زمان است که نشان می‌دهد زمان‌بندی تیزه‌های پیش‌سیناپسی و پس‌سیناپسی چگونه بر وزن سیناپس تأثیر می‌گذارد.

- اگر تیزه پیش‌سیناپسی قبل از پالس پس‌سیناپسی رخ دهد ($\Delta t < 0$)، وزن سیناپس افزایش می‌یابد (پتانسیاسیون).
- اگر تیزه پیش‌سیناپسی پس از پالس پس‌سیناپسی رخ دهد ($\Delta t > 0$)، وزن کاهش می‌یابد (افسردگی).

۳. مدل نورونی و شبیه‌سازی‌ها

در این مقاله، مدل نورونی پواسون استفاده شده است که در آن:

- تیزه‌های ورودی به صورت تصادفی و مستقل از یکدیگر تولید می‌شوند.
- نرخ شلیک خروجی $\lambda_{\text{out}}(t)$ بر اساس ورودی‌ها و وزن سیناپس‌ها تعیین می‌شود. این نرخ به صورت زیر تعریف شده است:

$$\lambda_{\text{out}}(t) = \lambda_0 + \sum_i J_i(t) \int_{-\infty}^t e^{(t-t')} S_{\text{in}}(t') dt'.$$

در این معادله:

کرنل پاسخ نورونی که نشان می‌دهد یک تیزه ورودی چگونه نرخ خروجی را تغییر می‌دهد. $e(t)$ نرخ شلیک خودبه‌خودی، λ_0

۴. ویژگی های کلیدی یادگیری

- نرمال سازی خود کار: مدل به طور طبیعی وزن سیناپس ها را بدون نیاز به تنظیم خارجی نرمال می کند. این نرمال سازی باعث می شود که نرخ شلیک نورو ن خروجی نیز در یک محدوده معقول باقی بماند.
- ایجاد ساختار: همبستگی بین تیزه های ورودی و خروجی به شکل گیری الگوهای پایدار در توزیع وزن سیناپسی منجر می شود.
- اثر نویز: نویز موجود در پالس های تصادفی تأثیر جزئی بر تغییرات وزن ها دارد و این اثر در حضور همبستگی های قوی کاهش می یابد.

۵. نتایج شبیه سازی

شبیه سازی های مقاله نشان دادند که:

- وزن سیناپس ها به طور طبیعی به مقدار ثابتی همگرا می شوند.
- توزیع وزن ها با توجه به همبستگی های ورودی، الگوهای خاصی ایجاد می کند.
- نویز در یادگیری تأثیر زیادی ندارد، به ویژه زمانی که تعداد سیناپس ها زیاد باشد.

شبیه‌سازی که ما انجام دادیم

```
[ ]: import numpy as np
import matplotlib.pyplot as plt

# Constants and Parameters
N = 50 # Total synapses
M1 = 25 # Group 1 (incoherent input)
M2 = 25 # Group 2 (coherent input)
J_initial = 0.1 # Initial weight for all synapses
t_total = 30 # Total simulation time in seconds
dt = 1.0 # Time step in seconds
num_steps = int(t_total / dt) # Number of simulation steps

# Learning parameters (from Appendix B)
h = 1e-5
w0 = h
wout = -1.0475 * h
w0_tilde = 6.043e-7 # Derived from Appendix B
v_in = 10.0 # Input frequency (Hz)
v_0 = 0.0 # Base output frequency (Hz)
J_star_w = -2e-2 # Fixed point for average weight
w_tilde = 4.75 * h * 1e-3
Q = 0.0004 * h

# Derived parameters
tau = 2e2 # Normalization time constant
tstr = 2.59e4 # Time constant for structure formation

# I synapse ها
poisson_matrix = np.random.poisson(v_in, (N, N))
```

و ۶ و ۸ و ۹ در مقاله را شبیه‌سازی کردیم.

```
def u(s):
    A_plus = 1
    A_minus = 1
    tau_plus = 1000
    tau_minus = 20000
    tau_syn = 1000
    if s < 0:
        return h*(A_plus*(1 - (s / tau_plus)) + A_minus*(1 - (s / tau_minus))) * np.exp(-s / tau_syn)
    else:
        return -h*(A_minus * np.exp(-s / tau_minus) + A_plus * np.exp(-s / tau_plus))

print(np.sum(u(num_steps)))

# Initialize weights and output arrays
weights = np.full((N, num_steps), J_initial)
v_out = np.zeros(num_steps)
J_w = np.zeros(num_steps)
J_1 = np.zeros(num_steps) # Average weights for group M1
J_2 = np.zeros(num_steps) # Average weights for group M2

# Simulation loop
for t in range(1, num_steps):
    J_w[t-1] = np.clip(J_w[t-1], -1.0, 1.0) # Clamp to avoid instability

    # Compute v_out before weight updates
    v_out[t] = v_0 + N * J_w[t-1] * v_in
    for j in range(N):
        for i in range(N):
```

```

jupyter Untitled20 Last checkpoint: 51 seconds ago
File Edit View Run Kernel Settings Help
JupyterLab Python 3 (ipykernel)

# Compute v_out before weight updates
v_out[t] = v_0 + N * z_av(t - 1) * v_in
for j in range(N):
    for i in range(N):
        v_in_i = poisson_matrix[j, i]
        v_in_i = v_in_i if i < M1 else -v_in_i # Group M1/M2 differentiation
        d3 = (
            w_in * v_in
            + w_out * v_out[0]
            + w_tilde * v_in * v_out[0]
            - N * (Q + w_out * v_in + w_tilde * v_in * v_in)
        )
        weights[i, t] = weights[i, t - 1] + dt * d3

# Clamp weights to avoid runaway growth
weights[i, t] = np.clip(weights[i, t], -0.1, 0.2) # Expected range

# Update group averages and overall average
z_1[t] = np.mean(weights[M1, t])
z_2[t] = np.mean(weights[M2, t])
z_av[t] = np.mean(weights[:, t])

# Plot Results
time = np.linspace(0, t_total, num_steps)

plt.figure(figsize=(10, 6))
plt.plot(time / 1e4, z_av, label='z_av (all synapses)', linestyle='--', color='blue')
plt.plot(time / 1e4, z_1, label='z_1 (group M1)', color='red')
plt.plot(time / 1e4, z_2, label='z_2 (group M2)', color='green')
plt.axhline(z_star_av, linestyle='dotted', color='black', label='z*_av (fixed point)')
plt.xlabel('Time (10^4 s)', fontsize=12)
plt.ylabel('Synaptic Heights', fontsize=12)

# From weights
time = np.linspace(0, t_total, num_steps)

plt.figure(figsize=(10, 6))
plt.plot(time / 1e4, z_av, label='z_av (all synapses)', linestyle='--', color='blue')
plt.plot(time / 1e4, z_1, label='z_1 (group M1)', color='red')
plt.plot(time / 1e4, z_2, label='z_2 (group M2)', color='green')
plt.axhline(z_star_av, linestyle='dotted', color='black', label='z*_av (fixed point)')
plt.xlabel('Time (10^4 s)', fontsize=12)
plt.ylabel('Synaptic Heights', fontsize=12)
plt.title('Temporal Evolution of Synaptic Heights', fontsize=14)
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# Insets: Weight Distributions at Specific Times
specific_times = [1e4, 1e4, 2.93e4, 7e4] # Times for weight distributions
for t_mark in specific_times:
    step = int(t_mark / dt)
    if step >= num_steps: # Skip if time exceeds simulation range
        print(f"Skipping t_mark={t_mark} as it exceeds simulation time.")
        continue
    plt.figure(figsize=(4, 3))
    plt.hist(weights[:, step], bins=20, color='gray', edgecolor='black')
    plt.title(f'Weight Distribution at t = {int(t_mark)} s')
    plt.xlabel('Weight')
    plt.ylabel('Frequency')
    plt.grid(True)
    plt.tight_layout()
    plt.show()

# Generate an array of s values for plotting
s_values = np.linspace(-30000, 30000, 1000) # Range of s values

```

```

jupyter Untitled20 Last checkpoint: 1 minute ago
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)

# Insets: Weight Distributions at Specific Times
specific_times = [1e3, 1e4, 2.5e4, 7e4] # Times for weight distributions
for t_mark in specific_times:
    step = int(t_mark / dt)
    if step > num_steps: # Skip if time exceeds simulation range
        print(f"Skipping t_mark={t_mark} as it exceeds simulation time.")
        continue
    plt.figure(figsize=(6, 3))
    plt.hist(weights[step, :], bins=20, color='gray', edgecolor='black')
    plt.title(f'Weight Distribution at t = {int(t_mark)} s')
    plt.xlabel('weight')
    plt.ylabel('frequency')
    plt.grid(True)
    plt.tight_layout()
    plt.show()

# Generate an array of s values for plotting
s_values = np.linspace(30000, 50000, 1000) # Range of s values

# Compute W(s) for all s values
W_values = np.array([W(s) for s in s_values])
# Plot the function W(s)
plt.figure(figsize=(8, 6))
plt.plot(s_values, W_values, label='W(s)', color='purple')
plt.axhline(0, color='black', linestyle='--', linewidth=0.8)
plt.axvline(0, color='black', linestyle='--', linewidth=0.8)
plt.xlabel('s (time difference)')
plt.ylabel('W(s)')
plt.title('Plot of the STDP Function W(s)')
plt.legend()
plt.grid(True)
plt.show()

jupyter Untitled20 Last checkpoint: 13 minutes ago
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)

# Parameters for the simulation from the paper
N = 50 # Total number of synapses
J_init = 0.1 # Initial weight (upper bound g = 0.1)
J_target = 0.02 # Target average weight J*
tau_av = 200 # Normalization time constant in seconds

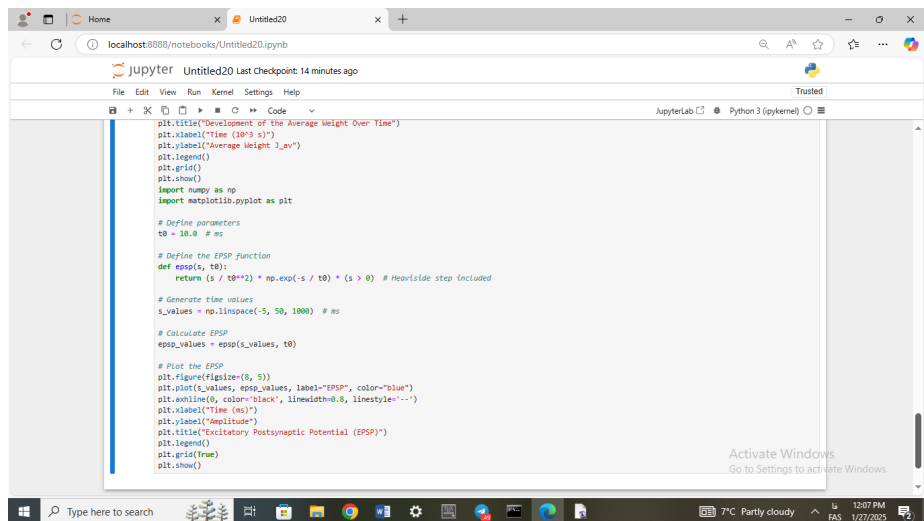
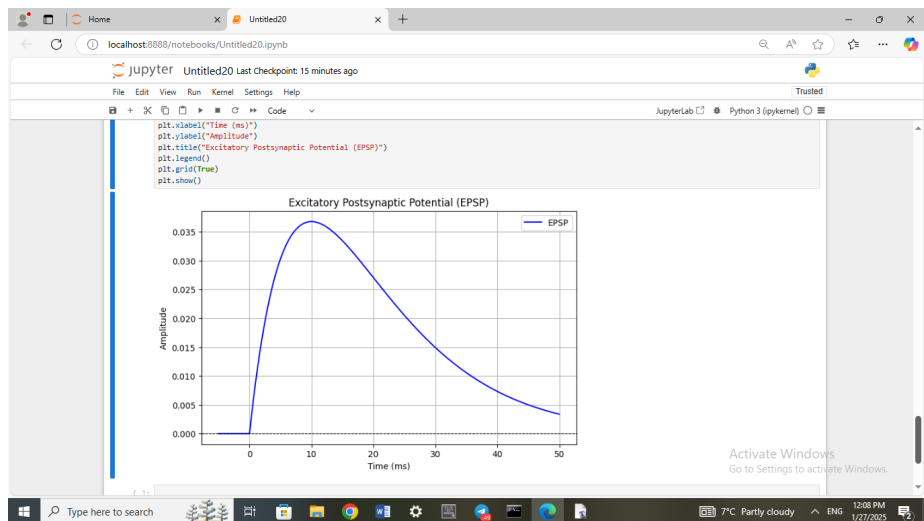
# Time settings
t_max = 1000 # Max simulation time in seconds
dt = 1 # Time step in seconds
time = np.arange(0, t_max, dt)

# Different initial conditions for weight distributions
J_initial_conditions = [
    0.0,
    0.01,
    0.025 * J_target,
    0.05,
    J_init,
]

# Function to simulate weight normalization
def simulate_weight_normalization(J, J_target, tau, t_steps):
    J = np.zeros_like(t_steps, dtype=float)
    J[0] = J
    for t in range(1, len(t_steps)):
        J[t] = J[t - 1] + (J_target - J[t - 1]) * (dt / tau)
    return J

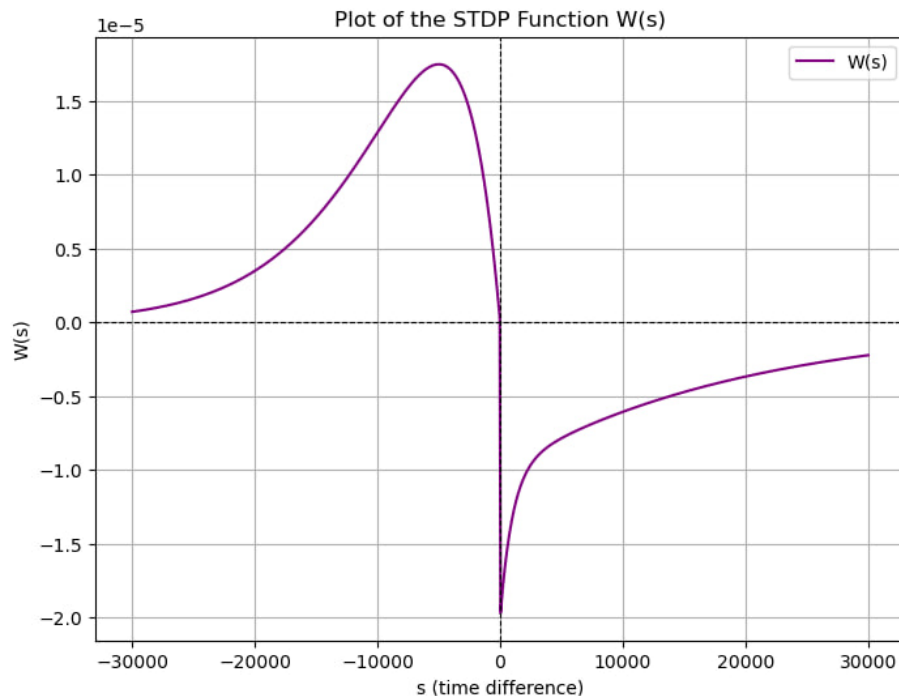
# Run simulations for different initial conditions
plt.figure(figsize=(8, 5))
for J0 in J_initial_conditions:
    J_values = simulate_weight_normalization(J0, J_target, tau_av, time)
    plt.plot(time / 1000, J_values, label=f'J_init = {J0:.3f}')

```

fig⁴

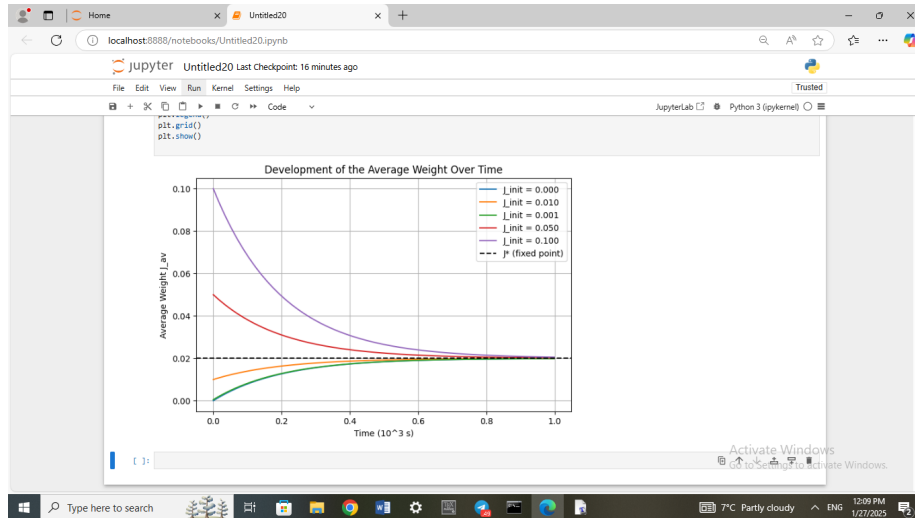
شکل ۴ نشان‌دهنده تغییرات پتانسیل پست‌سیناپسی است: وقتی اسپایک هنوز نرسیده ($s < 0$)، پتانسیل صفر است. در یک لحظه خاص ($s = t_0$) به حداکثر می‌رسد. سپس به آرامی و به صورت نمایی کاهش می‌یابد. نتیجه: این مدل نشان می‌دهد که نورون‌ها چگونه به ورودی‌ها واکنش نشان می‌دهند و برای بررسی یادگیری عصبی مفید است.

fig 6



شکل ۶ "پنجره یادگیری" را نشان می‌دهد که توضیح می‌دهد تغییرات وزن سیناپسی چگونه به زمان‌بندی اسپایک‌های پیش‌سیناپسی و پست‌سیناپسی وابسته است: اگر اسپایک پیش‌سیناپسی قبل از اسپایک پست‌سیناپسی رخ دهد، وزن سیناپس افزایش می‌یابد. اگر اسپایک پیش‌سیناپسی بعد از اسپایک پست‌سیناپسی رخ دهد، وزن سیناپس کاهش می‌یابد. اگر فاصله زمانی خیلی زیاد باشد، تأثیر یادگیری به صفر می‌رسد. نتیجه: این پنجره یادگیری توضیح می‌دهد که چگونه زمان‌بندی دقیق اسپایک‌ها باعث تقویت یا تضعیف سیناپس‌ها می‌شود و به شکل‌گیری شبکه‌های عصبی کمک می‌کند.

fig8

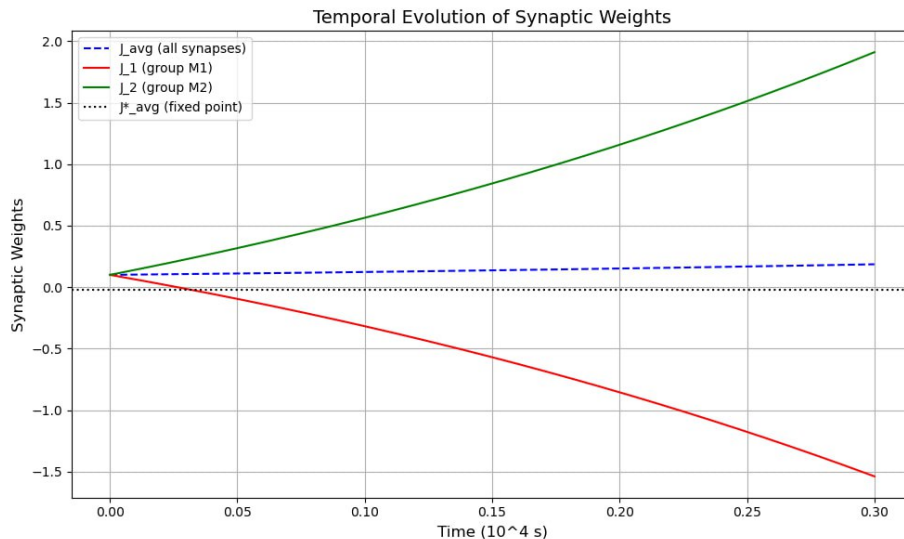


شکل ۸ تکامل وزن سیناپسی متوسط J_{av} را به عنوان تابعی از زمان t نشان می‌دهد. شبیه‌سازی‌ها با پنج مقدار اولیه مختلف برای J_{av} آغاز شده‌اند. ویژگی‌ها:

- در تمام موارد، J_{av} به سمت مقدار ثابت J_{av}^* همگرا می‌شود.
- مقدار ثابت J_{av}^* برابر با 2.03×10^{-2} است.
- زمان همگرایی به این مقدار، با یک ثابت زمانی $t_{av} = 2.03 \times 10^2$ ثانیه توصیف شده است.

نتیجه: این شکل نشان می‌دهد که با وجود شرایط اولیه متفاوت، وزن‌های سیناپسی در طول زمان نرمال می‌شوند و به یک نقطه تعادل پایدار همگرا می‌شوند. این رفتار تأثیر قوانین یادگیری هیبیان را در پایدارسازی شبکه‌های عصبی تأیید می‌کند.

fig ۹



نمودار نشان می‌دهد که:

- گروه M_1 (L_1): وزن‌های سیناپسی در این گروه کاهش پیدا می‌کند. یعنی سیناپس‌ها ضعیف‌تر می‌شوند.
- گروه M_2 (L_2): وزن‌های سیناپسی در این گروه افزایش پیدا می‌کند. یعنی سیناپس‌ها قوی‌تر می‌شوند.
- میانگین وزن‌ها (L_{avg}): ثابت است، یعنی کل سیستم تعادل خود را حفظ می‌کند.

این رفتار با قانون یادگیری هببیان (Hebbian) همخوانی دارد:

- سیناپس‌هایی که نورون‌هایشان هماهنگ فعال می‌شوند، تقویت می‌شوند.
- سیناپس‌هایی که فعالیت هماهنگی ندارند، تضعیف می‌شوند.

نتیجه:

- یک گروه (M_2) سیناپس‌های قوی‌تر می‌سازد.
- گروه دیگر (M_1) سیناپس‌هایش را از دست می‌دهد.
- اما سیستم به طور کلی پایدار می‌ماند.

چالش‌ها و مشکلات

ما در شبیه‌سازی فیگور ۹ به مشکلاتی خوردیم و برایمان کار سختی بود. ابتدا نمی‌توانستیم تیزه‌های تصادفی بسازیم و صرفاً با استفاده از معادلات و پارامترهای داخل مقاله سعی کردیم معادله دیفراسیل را حل کنیم و نمودار را بکشیم. اما بعد PySpike را پیدا کردیم و با استفاده از آن توانستیم فیگور ۹ را شبیه‌سازی کنیم.